DATA SCIENCE

Jupyter Notebooks & Anaconda

Master the notebook workflow, kernels, and IPython magics alongside conda environments and reproducible, shareable Python setups.

jupyter · anaconda · conda · jupyterlab · environments · reproducibility · data-science

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. 1. What They Are and Why They Matter
2. 2. Setup and Getting Started
3. 3. The Notebook Document and Cell Workflow
4. 4. IPython Magics and Shell Access
5. 5. Kernels and JupyterLab
6. 6. Conda: Environments and Package Management
7. 7. conda vs pip vs venv
8. 8. Cheat-Sheet Reference
9. 9. Best Practices and Common Pitfalls
10. 10. Interview Questions & Answers
11. 11. Further Learning

Jupyter and Anaconda are the two tools most data scientists launch first. **Jupyter** gives you a literate, cell-by-cell programming surface where code, output, math, and prose live in one document — the natural habitat for exploration, teaching, and reproducible analysis. **Anaconda** (and its lean cousin Miniconda) gives you a cross-platform package and environment manager that installs not just Python libraries but the compiled C/C++/Fortran stacks (MKL, CUDA, GEOS) they depend on, so your project runs the same on your laptop, a colleague's Mac, and a CI server. Learn the notebook workflow and conda environments well and you eliminate the two biggest sources of friction in data work: slow iteration and "it works on my machine".

1. What They Are and Why They Matter

These are complementary layers, not competitors. Anaconda sits *underneath* and provisions the Python interpreter plus every binary dependency; Jupyter runs *on top* and gives you an interactive editing experience backed by a language **kernel**. A healthy mental model separates three moving parts: the **front-end** you type into (the browser UI), the **kernel** that actually executes code and holds state, and the **environment** (a conda env) that supplies the packages the kernel imports.

ANACONDA / CONDA

A package *and* environment manager. Resolves and installs binary dependencies (not just Python), creates isolated environments, and reproduces them across OSes from a single file.

JUPYTER NOTEBOOK / LAB

A client that renders a `.ipynb` document and talks to a kernel over a messaging protocol. JupyterLab is the modern IDE-style interface; classic Notebook is the simpler one.

KERNEL

The process that runs your code and keeps variables alive between cells. IPython is the default Python kernel; kernels exist for R, Julia, Scala, and dozens more.

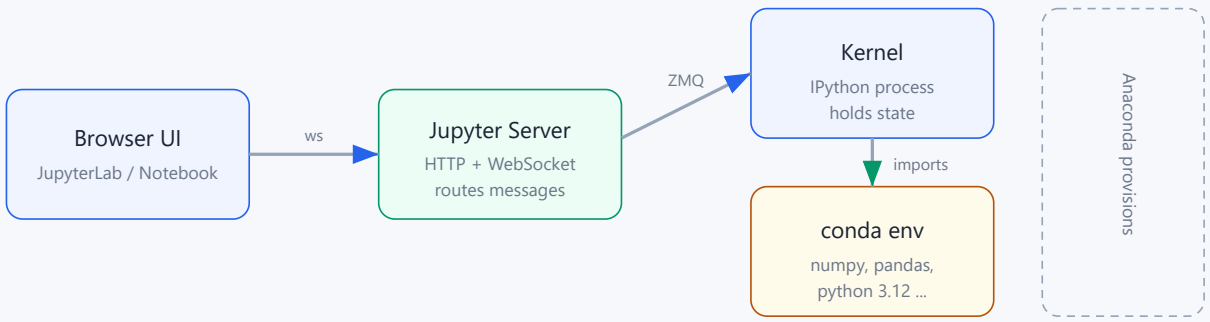


Fig 1. The four moving parts: UI talks to the Jupyter Server over WebSocket; the server relays execution requests to a kernel over ZeroMQ; the kernel imports packages from the conda environment it was launched in.

> **The single most useful insight**
> A notebook's output depends on *which kernel* ran it, and that kernel belongs to *one* environment. 90% of "ModuleNotFoundError even though I pip-installed it" problems are a mismatch: you installed into env A but the notebook is running env B's kernel.

2. Setup and Getting Started

Prefer **Miniconda** (just conda + Python) over the full Anaconda Distribution unless you want the ~3 GB bundle of pre-installed scientific packages. Even better for speed is **Miniforge**, which defaults to the community `conda-forge` channel and ships the fast `mamba` solver.

BASH

```bash
# --- Install Miniconda (Linux/macOS example) ---
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh
# restart your shell, then verify
conda --version          # conda 24.x or newer
conda init bash          # wire conda into your shell startup

# --- Create a project environment (never work in base!) ---
conda create -n ds python=3.12 numpy pandas jupyterlab -y
conda activate ds

# --- Launch ---
jupyter lab              # opens http://localhost:8888 in your browser
# or the classic interface:
jupyter notebook
```

> **Modern conda is fast**
> Since conda 23.10 the lightning-fast `libmamba` solver is the default. If you are on an older install, enable it once: `conda config --set solver libmamba`. Dependency resolution that took minutes now takes seconds.

To make a specific environment selectable as a kernel inside Jupyter, register it with `ipykernel`:

BASH

```
conda activate ds
conda install ipykernel -y
# register this env so it appears in the Jupyter kernel picker
python -m ipykernel install --user --name ds --display-name "Python (ds)"
```

3. The Notebook Document and Cell Workflow

A notebook is a JSON file (`.ipynb`) containing an ordered list of **cells**. Each cell is one of three types: `code` , `markdown` , or `raw` . Code cells send their source to the kernel and capture whatever the kernel returns — text, tables, images, HTML, interactive widgets.

The execution model (and its #1 trap)

Cells share a **single, persistent namespace** in the kernel. The number in `[ ]` to a cell's left is its *execution count* — the order in which it was actually run, which need not match top-to-bottom order on screen. This is the source of the notorious "hidden state" bug.

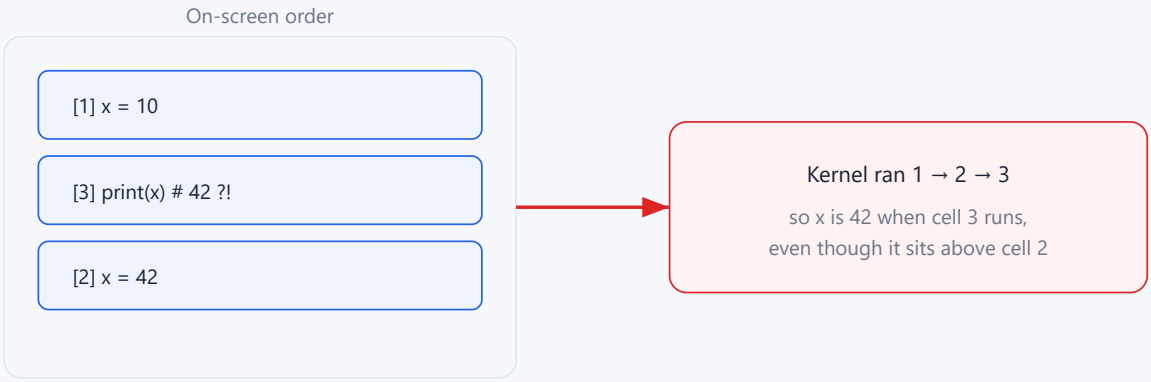


Fig 2. Execution count vs. visual order. The kernel remembers state across runs, so out-of-order execution yields results you cannot reproduce by reading top to bottom.

✕ **Always validate with "Restart & Run All"**
Before you trust, share, or commit a notebook, run **Kernel → Restart Kernel and Run All Cells**. If it fails or gives different numbers, your notebook had hidden state and is not reproducible.

Essential keyboard shortcuts

Jupyter has two modes: **command mode** (blue bar, acts on cells — press `Esc`) and **edit mode** (green bar, type into a cell — press `Enter`).

Shortcut	Mode	Action
`Shift`+`Enter`	any	Run cell, select cell below
`Ctrl`+`Enter`	any	Run cell, stay put
`Alt`+`Enter`	any	Run cell, insert new cell below
`A` / `B`	command	Insert cell above / below
`D` `D`	command	Delete cell
`M` / `Y`	command	Convert to markdown / code
`Z`	command	Undo cell deletion
`0` `0`	command	Restart the kernel
`Shift`+`M`	command	Merge selected cells

4. IPython Magics and Shell Access

The IPython kernel adds **magic commands**: special directives prefixed with `%` (line magic, affects one line) or `%%` (cell magic, affects the whole cell). They are not Python — they are kernel features for timing, profiling, loading code, and switching languages.

PYTHON

```
# Line magics (single %)
%timeit sum(range(1000))        # statistically robust micro-benchmark
%who                            # list variables defined in the namespace
%matplotlib inline              # render matplotlib figures in the output
%load_ext autoreload            # auto-reimport edited modules
%autoreload 2
%pwd                            # print working directory

# Cell magics (double %%)
%%timeit                        # time the WHOLE cell
total = 0
for i in range(10000):
    total += i

%%bash                          # run the cell as a shell script
ls -la
echo "hello from the shell"

%%writefile helper.py           # dump cell contents to a file
def greet(name):
    return f"Hi, {name}"
```

You can also run shell commands inline with `!`, and capture their output into a Python variable:

PYTHON

```
!pip install rich               # one-off shell command
files = !ls *.csv               # capture stdout into a Python list
print(files)

# Interpolate Python variables into shell with {}
name = "data.csv"
!head -n 3 {name}
```

⚠ **Use %pip and %conda, not !pip**

Inside a notebook, prefer the magics `%pip install <pkg>` and `%conda install <pkg>`. They install into the *exact environment the running kernel uses*. Plain `!pip` may resolve to a different Python on your `PATH`, which is how packages "disappear".

Magic	Purpose
`%timeit` / `%%timeit`	Benchmark a line / cell with multiple runs
`%time` / `%%time`	Time a single execution (wall + CPU)
`%prun` , `%lprun`	Function-level and line-level profiling
`%debug` , `%pdb on`	Post-mortem / automatic debugger on exceptions
`%run script.py`	Execute an external script in the namespace
`%env`	Get / set environment variables
`%store`	Persist a variable across kernel restarts
`%lsmagic`	List every available magic

5. Kernels and JupyterLab

The kernel is a separate OS process. The front-end never runs your code directly — it sends *execute_request* messages and renders the *display_data* and *stream* messages that come back. This decoupling is why you can interrupt a runaway loop (`I` `I`), restart a hung kernel, or even connect a remote kernel to a local UI.

JupyterLab vs. classic Notebook vs. the ecosystem

Tool	What it is	Reach for it when
JupyterLab	Full IDE: tabs, file browser, terminals, extensions, side-by-side notebooks	Default modern choice for serious work
Jupyter Notebook 7	The classic single-document UI, now rebuilt on JupyterLab components	You want a lightweight, focused view
VS Code / Cursor	Editor with native `.ipynb` support + debugger + Git	You live in an editor and want notebooks beside normal code
Google Colab	Hosted notebooks with free GPUs	No local setup; quick GPU experiments
JupyterHub	Multi-user server for teams/classes	Centralised, shared compute

A worked mini-example: exploratory data analysis

Here is the archetypal notebook flow — load, inspect, visualise — the kind of cell sequence you would type live in an interview or a real analysis.

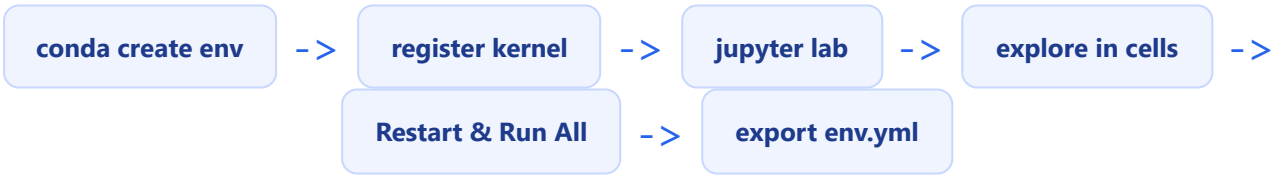
PYTHON

```python
# Cell 1 — setup
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

# Cell 2 — load and peek
df = pd.read_csv("sales.csv", parse_dates=["date"])
df.head()                       # rich HTML table in the output

# Cell 3 — quick profile
df.describe()
df.isna().sum()                 # missing-value audit

# Cell 4 — a visual, rendered inline
monthly = df.set_index("date")["revenue"].resample("ME").sum()
monthly.plot(figsize=(9, 4), title="Monthly revenue")
plt.ylabel("USD")
plt.tight_layout()
```



6. Conda: Environments and Package Management

An environment is a self-contained directory holding a specific Python interpreter and a specific set of packages. Isolation means Project A can use pandas 1.5 while Project B uses pandas 2.2, with zero conflict. Never install into `base` — keep it minimal and create one env per project.

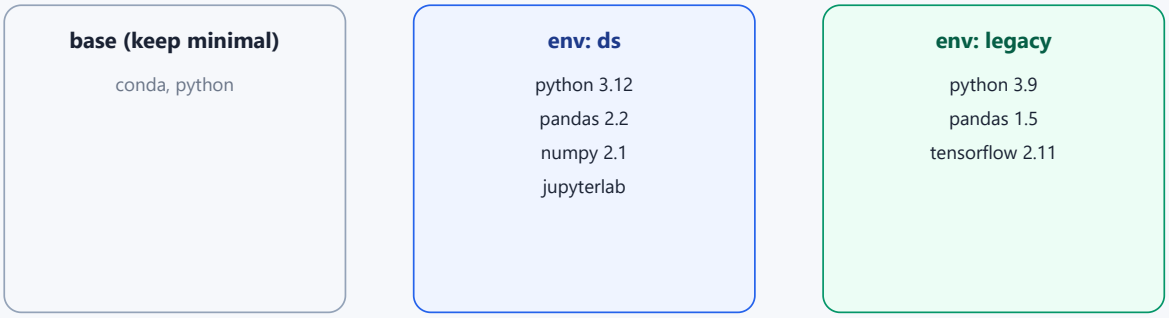


Fig 3. Isolated environments. Each project gets its own interpreter + pinned packages; incompatible versions coexist peacefully side by side.

BASH

```bash
# Lifecycle of a conda environment
conda create -n ds python=3.12          # create
conda activate ds                       # enter
conda install pandas scikit-learn       # install from default/conda-forge
conda install -c conda-forge polars     # install from a specific channel
conda list                              # what's installed
conda env list                          # all environments on the machine
conda update pandas                     # upgrade one package
conda remove numpy                      # uninstall
conda deactivate                        # leave
conda env remove -n ds                  # delete entirely

# Faster: use mamba as a drop-in replacement
mamba install pytorch -c pytorch
```

Reproducibility: environment files

The payoff of conda is sharing an exact environment. There are two export styles — a human-authored spec and a fully pinned lock.

YAML

```yaml
# environment.yml — the spec you hand-edit and commit to Git
name: ds
channels:
  - conda-forge
dependencies:
  - python=3.12
  - pandas≥2.2
  - scikit-learn
  - jupyterlab
  - pip
  - pip:                      # pip-only deps live in a nested block
      - some-pypi-only-package==1.4.0
```

BASH

```bash
# Recreate the environment anywhere from the spec
conda env create -f environment.yml
conda activate ds

# Update an existing env after editing the file
conda env update -f environment.yml --prune   # --prune removes dropped deps

# Fully pinned, exact export (good for a snapshot; OS-specific!)
conda env export > environment.lock.yml

# Cross-platform explicit spec (reproducible, no builds)
conda env export --from-history > environment.yml
```

✦ **--from-history is the portable export**

A plain `conda env export` pins exact build strings tied to your OS and will often fail to recreate on a different platform. `--from-history` records only the packages *you* explicitly asked for, producing a clean, cross-platform file.

7. conda vs pip vs venv

These overlap but solve different problems. **venv** isolates Python packages only. **pip** installs from PyPI into whatever environment is active. **conda** is both an environment manager *and* a package manager that also handles non-Python binaries. They can be combined — but with rules.

Dimension	conda	pip + venv
Manages environments	Yes (`conda create`)	venv does; pip does not
Package source	conda channels (conda-forge, defaults)	PyPI
Non-Python deps	Yes — CUDA, MKL, GDAL, compilers	No (relies on prebuilt wheels)
Language scope	Any language	Python only
Dependency solver	Full SAT solver (libmamba)	Resolver, less strict about binary ABI
Breadth of packages	Large but < PyPI	Everything on PyPI
Best for	Data science, GPU, geospatial, mixed stacks	Pure-Python apps, web, lightweight tools

> ⚠ **The golden rule for mixing conda and pip**
> Install as much as possible with conda *first*, then use `pip` only for what conda lacks — and do it *last*. Installing with conda after pip can silently clobber pip-managed files. Record pip deps in the `pip:` block of `environment.yml` so the environment stays reproducible.

The modern pure-pip world has also matured. Tools like **uv** (an extremely fast Rust-based resolver/installer) and **Poetry** or **PDM** manage pip-based projects with real lock files. For a pure-Python service, uv + venv is often faster and leaner than conda; for a GPU/compiled scientific stack, conda still wins.

BASH

```bash
# The venv + pip workflow (no conda)
python -m venv .venv              # create an env in ./.venv
source .venv/bin/activate         # (Windows: .venv\Scripts\activate)
pip install -r requirements.txt   # install pinned deps
pip freeze > requirements.txt     # snapshot exact versions

# The modern fast alternative: uv
uv venv                           # create .venv instantly
uv pip install pandas jupyterlab  # resolve + install in a flash
uv pip compile requirements.in -o requirements.txt  # lock
```

8. Cheat-Sheet Reference

conda command reference

Command	Does
`conda create -n NAME python=3.12`	Create a new environment
`conda activate NAME` / `deactivate`	Enter / leave an environment
`conda install -c conda-forge PKG`	Install from a channel
`conda list`	List installed packages
`conda env list`	List all environments
`conda env export --from-history > environment.yml`	Portable export
`conda env create -f environment.yml`	Recreate from file
`conda clean --all`	Free disk by removing caches/tarballs
`conda config --show channels`	Inspect channel configuration

Jupyter CLI reference

Command	Does
`jupyter lab`	Launch JupyterLab
`jupyter notebook`	Launch classic Notebook
`jupyter kernelspec list`	Show registered kernels
`python -m ipykernel install --user --name NAME`	Register current env as a kernel
`jupyter nbconvert --to html nb.ipynb`	Export to HTML / PDF / script
`jupyter nbconvert --to notebook --execute nb.ipynb`	Run a notebook headless (CI)
`jupyter lab --no-browser --port 8889`	Headless server (remote work)

9. Best Practices and Common Pitfalls

ONE ENV PER PROJECT

Create a named environment per repo and commit an `environment.yml`. Keep `base` clean so a broken install never bricks your whole conda.

NOTEBOOKS EXPLORE, MODULES SHIP

Prototype in a notebook, then move stable logic into importable `.py` files and `import` them back. Notebooks are not a good home for production code.

RESTART & RUN ALL BEFORE TRUSTING

Make linear reproducibility a habit. A notebook that only works with hidden state is a liability.

STRIP OUTPUT BEFORE COMMITTING

Notebook JSON diffs are noisy. Use `nbstripout` or Jupytext to pair with a `.py` so Git reviews are readable.

✓ **Do**

Pin Python and major libraries (`python=3.12` , `pandas≥2.2`). Use `%pip` / `%conda` inside notebooks. Prefer `conda-forge` as a single consistent channel. Set a project README with the one-line recreate command.

✗ **Avoid**

Installing everything into `base` . Mixing many channels (channel priority conflicts). Running `pip` then `conda` in the wrong order. Committing a 50 MB notebook full of embedded plot output. Relying on cell execution order you can't reproduce.

Show: diagnosing "ModuleNotFoundError" in a notebook

10. Interview Questions & Answers

What is the difference between a Jupyter front-end, a kernel, and an environment?

The front-end (browser UI or editor) renders the document and sends execution requests. The kernel is a separate process that actually runs the code and holds all variable state between cells.

The environment is the conda/venv that supplies the packages the kernel imports. A notebook's behaviour is determined by the kernel, and the kernel belongs to exactly one environment.

Why can a notebook produce results you cannot reproduce by reading it top to bottom?

Because cells share one persistent namespace and can be executed in any order. The number in `[ ]` is the execution count, not the position. If you run a lower cell, then edit and re-run an upper one, state carries over inconsistently. The fix is Restart Kernel and Run All, which forces linear, reproducible execution.

conda vs pip — when would you choose each?

conda when you need non-Python binaries or a complex scientific/GPU stack (CUDA, MKL, GDAL) and want one tool to manage both the environment and compiled dependencies. pip (with venv or uv) for pure-Python projects where you want the full breadth of PyPI and a lightweight footprint. They can be combined: conda first, pip last.

What is the correct way to combine conda and pip?

Install everything you can with conda first, then use pip only for packages conda lacks, and do pip last. Capture the pip packages in the `pip:` block of `environment.yml`. Running conda after pip can overwrite pip-managed files and corrupt the environment.

What is the difference between a line magic and a cell magic?

A line magic uses a single `%` and affects just that line (e.g. `%timeit func()`). A cell magic uses `%%` on the first line and applies to the entire cell (e.g. `%%timeit` times the whole block, `%%bash` runs the cell as shell). Magics are IPython kernel features, not Python syntax.

Why should you use `%pip install` instead of `!pip install` in a notebook?

`%pip` is routed to the exact Python interpreter the running kernel uses, so the package lands where your imports look. `!pip` shells out and may hit a different Python on your PATH, installing into the wrong environment — the classic cause of "I installed it but it's not found."

How do you make a notebook reproducible for a teammate?

Commit an `environment.yml` (ideally via `conda env export --from-history` for portability), pin Python and major library versions, strip output with nbstripout, and ensure the notebook passes Restart & Run All. The teammate recreates with `conda env create -f environment.yml` and selects the matching kernel.

What does `conda env export --from-history` give you that a plain export does not?

It records only the packages you explicitly requested, without OS-specific build strings. A plain `conda env export` pins exact builds tied to your platform, which frequently fails to recreate on a different OS. `--from-history` is the cross-platform, human-readable choice.

What is a kernel, concretely, and how do you add one for a specific conda env?

A kernel is an OS process implementing the Jupyter messaging protocol — for Python it's IPython. To expose a conda env as a kernel: activate it, `conda install ipykernel`, then `python -m ipykernel install --user --name myenv --display-name "Python (myenv)"`. It then appears in the kernel picker and the notebook will import from that env.

Why is `base` special, and why avoid installing into it?

base is the environment conda itself runs in. Installing project packages there risks dependency conflicts that can break the `conda` command, pollutes a shared space, and makes projects non-reproducible. Keep base minimal and always work in a dedicated per-project environment.

How would you run a notebook in CI or on a schedule?

Use `jupyter nbconvert --to notebook --execute nb.ipynb` (or Papermill, which also injects parameters) to run it headless and capture outputs/errors. Pair this with a locked environment so the run is deterministic. Papermill is the common choice for parameterised, scheduled notebook jobs.

What are the drawbacks of notebooks and how do you mitigate them?

Hidden state, poor diffability, weak testing, and encouragement of monolithic code. Mitigate with Restart & Run All, nbstripout/Jupytext for clean diffs, moving reusable logic into tested `.py` modules, and keeping notebooks focused on exploration and narrative rather than production pipelines.

11. Further Learning

- **Papermill** — parameterise and execute notebooks programmatically for scheduled jobs and reports.
- **Jupytext** — pair notebooks with plain `.py` / `.md` so Git diffs are clean and reviewable.
- **nbdev / nbconvert** — build libraries and docs directly from notebooks; export to HTML, PDF, slides.
- **JupyterLab extensions** — variable inspectors, Git integration, LSP code completion, debuggers.
- **uv and Poetry** — modern, fast dependency management for pure-Python projects as an alternative to conda.
- **Mamba / Miniforge** — faster solving and the conda-forge-first distribution; learn channel priority deeply.
- **JupyterHub and Binder** — multi-user deployments and one-click reproducible environments from a repo.